

[GRENOBLE INP - ESISAR]

PX222 - AUTO ELEC

Carnet de Bord

Contrôle de courant dans un circuit inductif

Antton Chevrier Lucas Fernandez Francis Ferguson

Période : Mars à juin 2026

Dernière mise à jour : 23 avril 2026

Table des matières

1	Mercredi 18 mars 2026	3
1.1	Modélisation des éléments du montage	3
1.1.1	Bobine	3
1.1.2	Amplificateur	3
1.1.3	Capteur de courant	4
1.1.4	Calcul de la boucle ouverte	4
1.2	Matériel et Sécurité	4
1.2.1	Liste du matériel prévu	4
1.2.2	Consignes de sécurité et protocoles	4
1.3	Pour la prochaine séance	5
2	Mardi 31 mars 2026	7
2.0.1	Analyse du signal de commande	9
2.1	Conclusion de séance	9
	Interséance	10

Introduction

L'objectif de ce projet est de réaliser un asservissement de courant dans un circuit inductif, au travers de la bobine étudiée en PX221. Nous utiliserons un microcontrôleur STM-32 pour réaliser l'asservissement numérique.

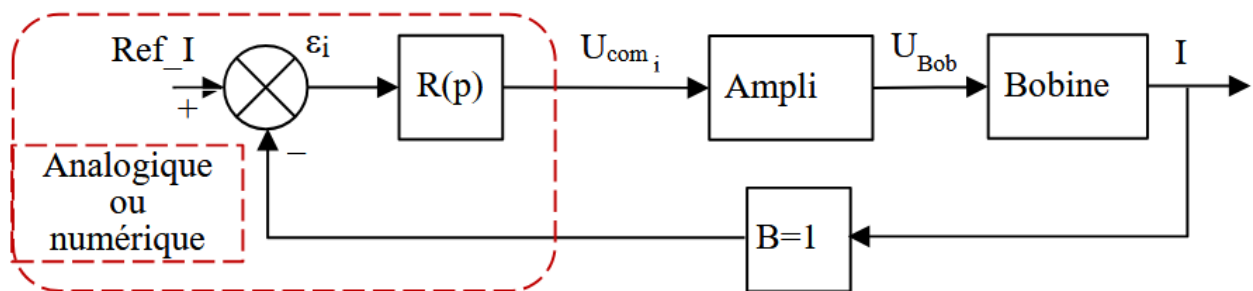


FIGURE 1 – Schéma bloc du montage à réaliser

Séance 1

Mercredi 18 mars 2026

1.1 Modélisation des éléments du montage

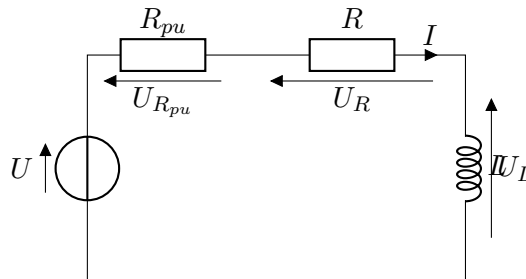
Pour commencer, nous avons modélisé les différents éléments du montage à l'aide de leurs fonctions de transfert respectives.

1.1.1 Bobine

Pour l'étude, nous retenons un **modèle série simple** de la bobine à noyau de fer (BNF). Ce modèle prend en compte l'inductance L représentant l'auto-induction et la résistance série R constituée par l'enroulement en cuivre.

Nous y ajoutons en série une résistance de puissance $R_{pu} = 12\ \Omega$ afin de diviser la constante de temps du système par 2. D'après les mesures du semestre précédent (PX221), nous savons que la résistance interne de la bobine est $R = 14\ \Omega$ et son inductance est $L = 1,2\ \text{H}$.

La résistance totale du circuit est donc : $R_{tot} = R + R_{pu} = 14 + 12 = 26\ \Omega$.



On obtient la fonction de transfert de la charge :

$$H_B(p) = \frac{I}{U} = \frac{H_0}{1 + \tau p} \quad \text{avec } H_0 = \frac{1}{R_{tot}} = \frac{1}{26} \text{ et } \tau = \frac{L}{R_{tot}} = \frac{1,2}{26} \approx 0,046 \text{ s}$$

1.1.2 Amplificateur

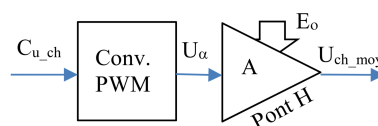


FIGURE 1.1 – Schéma bloc fonctionnel de l'amplificateur

L'amplificateur est un dispositif à découpage avec une structure en pont en H. Nous utilisons un **modèle moyen** pour l'asservissement. En effet, la fréquence de découpage ($f > 20\ \text{kHz}$) est fixée pour

être inaudible et est très supérieure à la constante de temps de la charge. Cela permet d'approximer la tension aux bornes de la bobine par sa valeur moyenne $U_{ch_{moy}}$. D'après le sujet, la tension de sortie est donnée par : $U_{ch_{moy}} = (2U_{com} - 1)E_0$. Le gain statique de l'amplificateur est alors défini par la dérivée :

$$K_A = \frac{dU_{Bob}}{dU_{com}} = 2E_0$$

1.1.3 Capteur de courant

La mesure du courant s'effectue via la résistance de puissance R_{pu} . Le signal traverse un diviseur de tension par 12 pour rester dans la plage de tension de l'électronique, puis un amplificateur d'instrumentation ($U201$). Ce montage permet d'obtenir une conversion directe au point de test $TP201$: $1\text{ V} \equiv 1\text{ A}$. On en déduit le gain de la chaîne de retour : $B = 1\text{ V/A}$.

1.1.4 Calcul de la boucle ouverte

La fonction de transfert en boucle ouverte (FTBO) du système complet est :

$$\begin{aligned} H_{BO}(p) &= R(p) \cdot K_A \cdot H_B(p) \cdot B \\ &= k \cdot (2E_0) \cdot \frac{1}{1 + \tau p} \cdot 1 \\ &= \boxed{\frac{2kE_0}{1 + \tau p}} \end{aligned}$$

1.2 Matériel et Sécurité

1.2.1 Liste du matériel prévu

Pour la mise en œuvre pratique de cet asservissement, nous utiliserons les équipements suivants :

- **Maquette PX222** : Elle servira de base pour le pont en H, les drivers de MOSFET et le conditionnement des signaux.
- **Bobine à Noyau de Fer (BNF)** : Identifiée en PX221, elle constituera notre charge inductive principale ($L \approx 1,2\text{ H}$).
- **Résistance de puissance ($12\ \Omega$)** : Elle sera placée en série pour réduire la constante de temps et servira également au capteur de courant.
- **Microcontrôleur STM32 (Nucleo-F103RB)** : Il sera utilisé pour l'acquisition des signaux (ADC) et la génération de la commande PWM numérique.
- **Appareillage de mesure** : Nous aurons besoin d'un oscilloscope numérique, d'un GBF pour la consigne de courant et de multimètres.
- **Alimentation de puissance** : Une source de tension continue réglable sera utilisée pour alimenter le pont en H (maximum 60 V).

1.2.2 Consignes de sécurité et protocoles

La manipulation de puissances importantes imposera le respect strict des consignes suivantes :

- **Protection électrique** : Toute tension supérieure à 50 V présentant un danger mortel, nous serons donc extrêmement attentifs lors des essais sous tension.
- **Sondes différentielles** : Pour éviter tout court-circuit via la terre de l'oscilloscope lors des mesures sur le réseau ou le pont flottant, l'usage de sondes différentielles sera obligatoire.
- **Risque thermique** : La résistance de puissance pourra atteindre des températures supérieures à 100°C ; nous éviterons tout contact direct durant et après les essais.

1.3 Pour la prochaine séance

Nous devons avoir terminé les calculs des fonctions de transfert en boucle ouvertes et fermée, ainsi que le calcul du correcteur proportionnel.

Interséance

Nous avons calculé la fonction de transfert en boucle fermée du système.

$$\begin{aligned} H_{BF}(p) &= \frac{H_{BO}(p)}{1 + H_{BO}(p)} \\ &= \frac{\frac{2kE_0}{R+R_{pu}} \frac{1}{1+\tau p}}{1 + \frac{2kE_0}{R+R_{pu}} \frac{1}{1+\tau p}} \\ &= \frac{K_{BO}}{1 + \tau p + K_{BO}} \quad \text{avec } K_{BO} = \frac{2kE_0}{R + R_{pu}} \\ &= \frac{\frac{K_{BO}}{1+K_{BO}}}{1 + \frac{\tau p}{1+K_{BO}}} \end{aligned}$$

On souhaite également une erreur statique de 5%. Or

$$\varepsilon = \frac{1}{1 + K_{BO}} = \frac{1}{1 + \frac{2kE_0}{R+R_{pu}}} = 0.05$$

donc

$$1 + \frac{2kE_0}{R + R_{pu}} = \frac{1}{0.05}$$

d'où

$$k = \left(\frac{1}{0.05} - 1 \right) \frac{R + R_{pu}}{2E_0}$$

En faisant l'application numérique avec $R + R_{pu} = 26 \, \Omega$ et $E_0 = 60 \, \text{V}$, on trouve $k \approx 4.1$

Séance 2

Mardi 31 mars 2026

Bien que théoriquement, le gain proportionnel de 4 calculé précédemment fonctionne en théorie, il ne sera pas réalisable en pratique. En effet, le signal de commande U_{com} issu du correcteur doit piloter un convertisseur PWM dont la plage d'entrée est strictement limitée entre 0 V et 1 V.

Pour une consigne unitaire de 1 A, un gain $k = 4,1$ entraînerait une commande $U_{com} = 4,1$ V dès le début de l'échelon, provoquant une saturation immédiate du rapport cyclique à 100 %. Cette saturation rendrait le système non-linéaire et invaliderait nos modèles de transfert.

Nous fixons donc $k = 1$ afin de garantir que, même pour une erreur maximale ($\varepsilon = 1$), la commande reste dans la plage linéaire (1 V). Pour compenser l'erreur statique tout en respectant cette contrainte, nous étudions un correcteur Proportionnel-Intégral (PI).

Pour le réglage du paramètre d'intégration ω_i , nous utilisons la méthode de **compensation de pôle**. L'objectif est d'annuler le pôle lent de la bobine ($\tau \approx 0,046$ s) en réglant la constante de temps du correcteur telle que $T_i = 1/\omega_i = \tau$.

Cela donne une valeur théorique de départ :

$$\omega_i = \frac{1}{\tau} = \frac{R_{tot}}{L} = \frac{26}{1,2} \approx 21,67 \text{ rad/s}$$

Nous utilisons pour cela un script matlab qui nous permet de faire varier ω_i , avec un gain $k = 1$ et de visualiser la réponse temporelle du système.

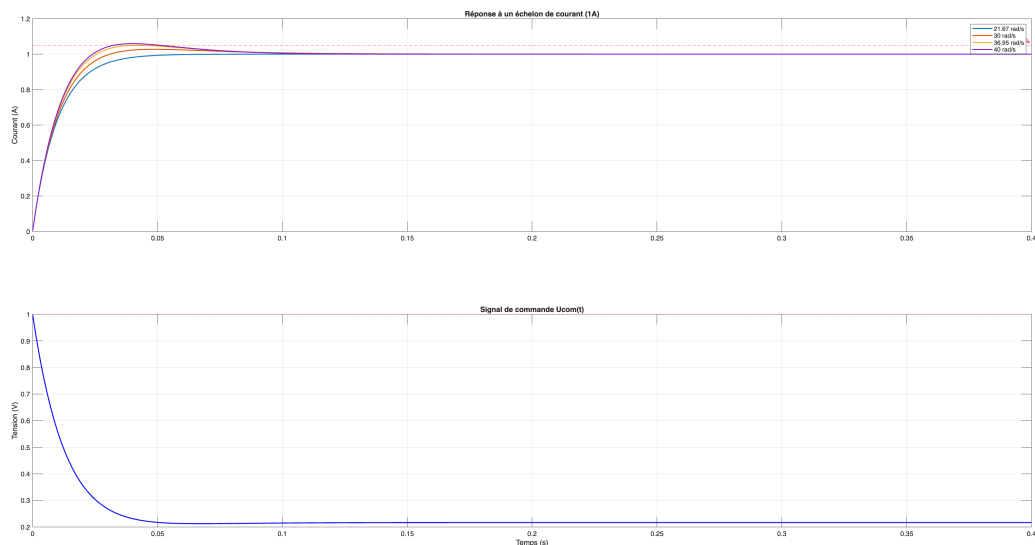


FIGURE 2.1 – Réponse temporelle du système pour différentes valeur de ω_i


```

1  % --- Parametres du systeme ---
2  Rs = 14;
3  Rext = 12;
4  Rtot = Rs + Rext;
5  L = 1.2;
6  tau = L / Rtot;
7  E0 = 60;
8  Ka = 2 * E0;
9  B = 1;
10
11 s = tf('s');
12 H_charge = (1/Rtot) / (1 + tau*s);
13 H = Ka * H_charge * B;
14
15 % --- Optimisation du PI ---
16 k = 1; % Gain fixe pour eviter la saturation
17 wi_tests = [21.67, 30, 36.95, 40]; % Inclut la compensation de pole
18
19 figure('Name', 'Reponse_PI_et_Commande');
20 for wi = wi_tests
21     R = k * (1 + wi/s);
22     FTBO = R * H;
23     FTBF = feedback(FTBO, 1);
24
25     % Analyse des performances
26     info = stepinfo(FTBF);
27     [y, t] = step(FTBF, 0.4);
28
29     subplot(2,1,1); plot(t, y, 'LineWidth', 1.5); hold on;
30     fprintf('wi=%0.2f | Depassement=%0.2f%% | Tr(5%%)=%0.3f s\n', ...
31             wi, info.Overshoot, info.SettlingTime);
32 end
33
34 % --- Mise en page graphique ---
35 subplot(2,1,1);
36 yline(1.05, 'r--', 'Limite_5%'); grid on;
37 title('Reponse_a_un_echelon_de_courant(1A)');
38 ylabel('Courant(A)');
39 legend(string(wi_tests) + " rad/s");
40
41 % Visualisation de la commande pour wi choisi
42 R_opt = k * (1 + 36.95/s);
43 Ucom = feedback(R_opt, H); % Transfert Consigne -> Commande
44 [u, t_u] = step(Ucom, 0.4);
45 subplot(2,1,2); plot(t_u, u, 'b', 'LineWidth', 1.5); hold on;
46 yline(1, 'r--', 'Saturation_PWM(1V)'); grid on;
47 title('Signal_de_commande_Ucom(t)');
48 ylabel('Tension(V)'); xlabel('Temps(s)');

```

Listing 2.1 – Script Matlab pour trouver le bon paramètre ω_i

L'analyse des courbes montre que la valeur $\omega_i = 36,95$ rad/s offre le meilleur compromis. Bien qu'elle soit supérieure à la valeur théorique de compensation, elle permet d'accélérer le système tout en restant sous la limite des 5 % de dépassement.

Les performances simulées sont les suivantes :

- **Dépassement** : 4,92 % (inférieur à la limite de 5 %).
- **Temps de réponse (5 %)** : environ 65 ms.
- **Erreur statique** : nulle (grâce à l'action intégrale).

2.0.1 Analyse du signal de commande

Le second graphique de la Figure 2.1 présente l'évolution de la tension de commande $U_{com}(t)$. Cette analyse est cruciale pour valider la faisabilité technologique de notre asservissement :

- **Respect de la plage linéaire** : On observe que pour le réglage optimal ($\omega_i = 36,95 \text{ rad/s}$), le signal débute à exactement 1 V sans jamais le dépasser. Cela confirme que notre choix $k = 1$ permet d'exploiter la dynamique maximale du convertisseur PWM (limité à 1 V d'après le sujet) sans jamais saturer le rapport cyclique à 100 %.
- **Effort de commande** : Le pic initial à 1 V illustre l'action proportionnelle qui fournit l'énergie nécessaire pour vaincre l'inertie de la charge inductive. La décroissance qui suit montre l'action du terme intégral qui ajuste finement la tension pour stabiliser le courant à 1 A, prouvant que le système ne force pas inutilement sur les composants une fois le régime établi.

2.1 Conclusion de séance

Pour la prochaine séance, nous devons nous renseigner sur l'utilisation et la programmation de carte STM32, notamment sur les différents ports disponibles les différentes façon de brancher les différents élément du montage à la carte.

Interséance

Suite aux simulations effectuées lors de la séance précédente, nous avons préparé l'implémentation du correcteur PI sur le microcontrôleur STM32 (Nucleo-F103R8) afin de réaliser l'asservissement numérique.

Stratégie de programmation

Pour garantir la précision temporelle de l'asservissement, nous avons choisi une programmation par accès direct aux registres. Les choix techniques sont les suivants :

- **PWM (TIM1_CH1 sur PA8)** : Configuration d'une porteuse à environ 22 kHz pour piloter le pont en H de la maquette.
- **Acquisition (ADC1_IN5 sur PA5)** : Lecture de l'image du courant convertie à 1 V/1 A par l'amplificateur d'instrumentation de la carte.
- **Régulation** : Implémentation de la loi de commande avec $k = 1$ et $\omega_i = 36,95 \text{ rad/s}$. Une saturation logicielle à 1 V est ajoutée pour protéger le système.

Code source concernant les registres et le setup de la carte

```
1  #include <stdint.h>
2
3  // --- ADRESSES REGISTRES STM32F103 ---
4  #define RCC_BASE      0x40021000
5  #define GPIOA_BASE    0x40010800
6  #define ADC1_BASE     0x40012400
7  #define TIM1_BASE     0x40012C00
8
9  // Horloges
10 #define RCC_APB2ENR    (*(volatile uint32_t *) (RCC_BASE + 0x18))
11 #define RCC_CFGR       (*(volatile uint32_t *) (RCC_BASE + 0x04))
12
13 // GPIOA
14 #define GPIOA_CRL      (*(volatile uint32_t *) (GPIOA_BASE + 0x00))
15 #define GPIOA_CRH      (*(volatile uint32_t *) (GPIOA_BASE + 0x04))
16
17 // ADC1 (PA5 est ADC_IN5 sur F103)
18 #define ADC1_SR        (*(volatile uint32_t *) (ADC1_BASE + 0x00))
19 #define ADC1_CR2       (*(volatile uint32_t *) (ADC1_BASE + 0x08))
20 #define ADC1_SQR3      (*(volatile uint32_t *) (ADC1_BASE + 0x34))
21 #define ADC1_DR        (*(volatile uint32_t *) (ADC1_BASE + 0x4C))
22
23 // TIM1 (PWM sur PA8)
24 #define TIM1_CR1        (*(volatile uint32_t *) (TIM1_BASE + 0x00))
25 #define TIM1_PSC        (*(volatile uint32_t *) (TIM1_BASE + 0x28))
26 #define TIM1_ARR        (*(volatile uint32_t *) (TIM1_BASE + 0x2C))
27 #define TIM1_CCR1       (*(volatile uint32_t *) (TIM1_BASE + 0x34))
28 #define TIM1_CCMR1      (*(volatile uint32_t *) (TIM1_BASE + 0x18))
```

```

29 #define TIM1_CCER      (*(volatile uint32_t *) (TIM1_BASE + 0x20))
30 #define TIM1_BDTR      (*(volatile uint32_t *) (TIM1_BASE + 0x44))
31
32 void setup(void) {
33     // 1. Activer horloges : GPIOA, ADC1, TIM1 et AFIO
34     RCC_APB2ENR |= (1 << 0) | (1 << 2) | (1 << 9) | (1 << 11);
35
36     // 2. Configurer PA8 en "Alternate Function Push-Pull" (PWM TIM1_CH1)
37     GPIOA_CRH &= ~(0xF << 0);
38     GPIOA_CRH |= (0xB << 0); // Speed 50MHz, AF Push-Pull
39
40     // 3. Configurer PA5 en mode Analogique (Entree mesure courant)
41     GPIOA_CRL &= ~(0xF << 20);
42
43     // 4. Configurer TIM1 pour PWM a 22kHz
44     TIM1_PSC = 0; // Pas de diviseur
45     TIM1_ARR = 3272; // 72MHz / 22kHz environ = 3272
46     TIM1_CCMR1 |= (0x6 << 4); // Mode PWM 1 sur CH1
47     TIM1_CCER |= (1 << 0); // Activer sortie CH1
48     TIM1_BDTR |= (1 << 15); // Main Output Enable (MOE) - Specifique
49     TIM1 TIM1_CR1 |= (1 << 0); // Lancer Timer
50
51     // 5. Configurer ADC1
52     ADC1_SQR3 = 5; // Selectionner canal 5 (PA5)
53     ADC1_CR2 |= (1 << 0); // Allumer ADC
54 }
55
56 uint32_t read_ADC(void) {
57     ADC1_CR2 |= (1 << 22); // Lancer conversion
58     while (!(ADC1_SR & (1 << 1))); // Attendre fin
59     return ADC1_DR;
60 }
61
62 void delay_ms(uint32_t ms) {
63     for (uint32_t i = 0; i < ms * 8000; i++) __asm__("nop");
64 }

```

Listing 2.2 – Implémentation du correcteur PI sur STM32F103

Code source concernant les calculs du correcteur PI et de la commande

```

1 // --- PARAMETRES DU CORRECTEUR PI ---
2 const float Kp = 1.0f; // Gain proportionnel
3 const float Wi = 36.95f; // Parametre d'integration
4 const float Ts = 0.001f; // Periode d'echantillonnage (1ms)
5 float integrale = 0.0f;
6 float consigne = 1.0f; // 1V ==> 1A
7
8 while (1) {
9     // --- 1. ACQUISITION ---
10    uint32_t val_adc = read_ADC();
11    // Conversion ADC (12 bits) en Volts : val * 3.3 / 4095
12    float courant_mesure = (float)val_adc * (3.3f / 4095.0f);
13
14    // --- 2. CALCUL DU PI ---
15    float erreur = consigne - courant_mesure;
16    integrale += erreur * Ts;
17

```

```

18 // Commande = Kp * (erreur + Wi * integrale)
19 float commande = Kp * (erreur + Wi * integrale);
20
21 // --- 3. SATURATION ET PROTECTION ---
22 // On bride la commande entre 0 et 1V pour rester en zone lineaire
23 if (commande > 1.0f) commande = 1.0f;
24 if (commande < 0.0f) commande = 0.0f;
25
26 // --- 4. ACTION (PWM) ---
27 // Le rapport cyclique est proportionnel a la commande (0V=0, 1V
    =100%)
28 // On ramene la commande 0-1V vers l'ARR du Timer (3272)
29 TIM1_CCR1 = (uint32_t)(commande * 3272.0f);
30
31 delay_ms(1); // Echantillonnage a 1kHz
32 }

```

Listing 2.3 – Implémentation du correcteur PI sur STM32F103